应用发现与管理核心

1. 课程目标

GUI 系统启动后，桌面上的图标是从哪里来的？为什么我们往 SD 卡里放一个文件夹，系统就能识别它？本节课我们将介绍 `AppManager` 的"应用发现"机制。

2. 核心源码分析

文件路径: `src/canmv/port/builtin_py/ybMain/main.py`

`AppManager` 中的 `scan_apps` 方法（约 689 行）负责扫描并加载应用。这个方法展示了如何通过 Python 的动态特性来实现插件化。

2.1 应用的存储结构

所有的应用都存放在 `/sdcard/apps/` 目录下。这是一个典型的**插件化**结构。

```
/sdcard/apps/
├── camera/           <-- 这是一个 App
|   ├── app.py        <-- 代码入口
|   └── icon.png      <-- 桌面图标
├── setting/          <-- 这是另一个 App
|   ├── app.py
|   └── icon.png
└── ...
```

2.2 动态加载原理

标准的插件（app）的加载流程如下：

1. **遍历目录**: 系统启动时，`AppManager` 会列出 `apps` 目录下的所有子文件夹。
2. **验证合法性**: 检查文件夹内是否有 `app.py`。
3. **导入模块**: 使用 Python 的 `import` 机制动态加载 `app.py`。
4. **实例化**: 每个 `app.py` 里都有一个 `App` 类，系统会创建它的实例，并将其添加到管理列表中。

2.3 应用注册与图标创建 (`register_app` & `create_app_icon`)

应用实例化后，会被添加到 `self.apps` 字典中。随后，`AppManager` 会调用 `create_app_icon` 来绘制桌面图标。
这个函数不仅仅是画一个图片，它还包含了精致的动画逻辑：

1. **颜色生成**:
   - 如果 App 自带 `icon`，则使用图片。
   - 如果没有图片但定义了 `color`，则自动生成渐变色背景。
   - 如果什么都没有，则从预设的 iOS 风格色板中轮询选择颜色。
2. **入场动画 (Entrance Animation)**:
   - 我们利用 `lv.anim_t` 创建了三个并发动画：图标从小变大 (`size_anim`)、图标淡入 (`opa_anim`)、文字淡入 (`label_anim`)。
   - **波浪效果**: `delay_ms = (row * self.icons_per_row + col) * 20`。通过根据行列位置计算延迟时间，让图标像波浪一样一个接一个地弹出。

2.4 翻页逻辑的实现 (`change_page`)

当应用数量超过一页时，或者用户滑动屏幕时，需要进行翻页。翻页的实现逻辑非常简洁有效：

1. **清空旧页面**:
    - 遍历主屏幕 (`self.home_screen`) 上的所有子对象。
    - 通过坐标判断子对象是否在“应用网格区域”内。
    - 如果是应用图标（而不是状态栏或 Dock 栏），直接 `delete()` 删除。

```
# 伪代码逻辑
for child in home_screen.children:
    if child.y >= grid_start_y:
        child.delete()
```

2. **创建新页面**:
    - 更新 `self.current_page` 变量。
    - 重新遍历所有 App，计算它们在新页面是否应该显示。
    - 如果应该显示，再次调用 `create_app_icon`，这也意味着每次翻页都会触发一次炫酷的图标弹出动画。

```
app_page = (app_index) // self.apps_per_page + 1
if app_page == page_number:
    self.create_app_icon(self.apps[app_name])
```